Deep Learning

Lecture 1 — The Machine Learning Toolbox: a Crash Course

Przemysław Witaszczyk

Jagiellonian University
Faculty of Physics, Astronomy and Applied Computer Science

What this course is

- This is a complete, self-contained course in **applied deep learning**: it is built on theory and developed through Python practice.
- Every result on these slides is either derived in full or explicitly labelled as quoted. The lecture decks are the primary learning object; no outside reading is required to follow the arguments.
- The framing is mathematical throughout:

The course in one sentence

Neural networks are **parameterized compositional function spaces**, trained by **gradient-based optimization**.

How the course is built

- Theory and code carry equal weight: each lecture derives its core mathematics in full, and runs short, verified programs whose outputs appear on the slides exactly as they were produced.
- Every core formula receives a hand-computable worked numeric, and every such hand computation has been verified in Python before being typeset.
- We build on two books, which serve as enrichment rather than as prerequisites: Géron, *Hands-On Machine Learning* (2025), and Raschka et al., *Machine Learning with PyTorch and Scikit-Learn* (2022).

The thirteen lectures

1. **The ML toolbox (today)**
2. Perceptron, linear models, gradient descent
3. The MLP and backpropagation
4. PyTorch in depth
5. Training deep nets I: signal propagation
6. Training deep nets II: optimizers, regularization
7. Convolutional networks
8. Sequences: RNNs, LSTM
9. Attention & the Transformer
10. Large language models
11. ViT, multimodal, graph neural networks
12. Generative models I: AE, VAE, GAN
13. Generative models II: diffusion + synthesis

Lectures 1–3 work with scikit-learn and NumPy only; PyTorch enters in Lecture 4 and remains the framework for the rest of the course.

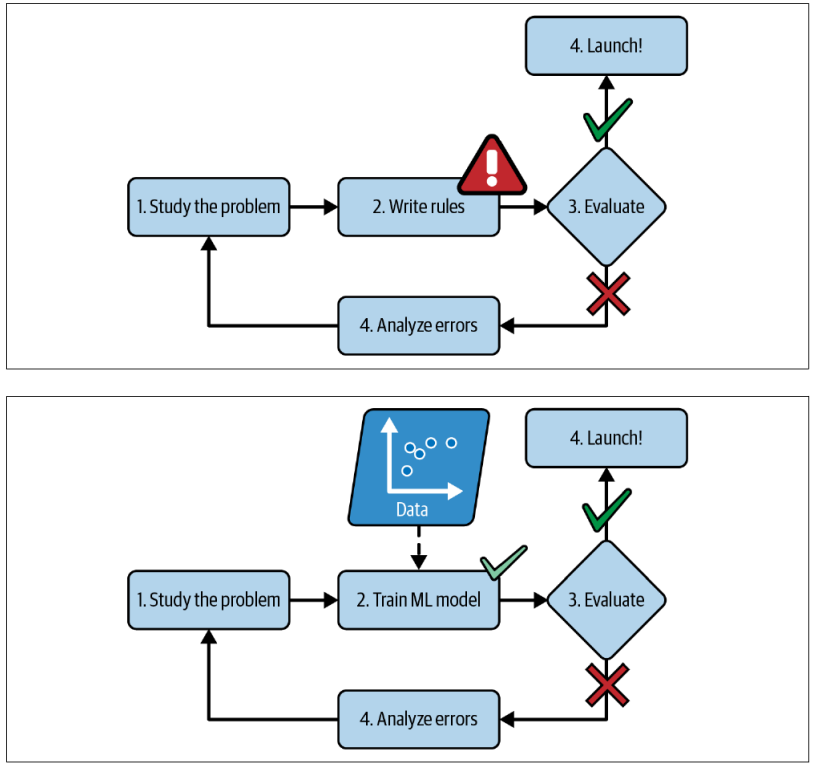
The plan for today: the toolbox before any neural network

- Deep learning *is* machine learning (ML); every methodological principle established today applies unchanged to the most advanced architecture of Lecture 13.
- This lecture establishes the foundations to which the entire course refers back. We state precisely what learning is; we establish **the dataset contract**, which fixes the roles of training, validation, and test data; and we derive the **bias–variance decomposition** in full, as the theory of generalization.
- We then fix **metrics** before training and compute them by hand, introduce **cross-validation** and the leakage-proof **pipeline discipline**, and close with one complete end-to-end project, carried out with real code and real numbers.
- Model internals are deliberately absent today. The first neuron appears in Lecture 2.

Symbols we will use (fixed for the whole course)

- Examples carry a superscript index: $(\mathbf{x}^{(i)}, y^{(i)})$ for $i = 1, \ldots, m$, where m is the number of examples and $\mathbf{x}^{(i)} \in \mathbb{R}^d$ collects d features.
- A **model** is a function $f_{\boldsymbol{\theta}}$ with parameters $\boldsymbol{\theta}$; its prediction is written $\hat{y} = f_{\boldsymbol{\theta}}(\mathbf{x})$.
- $\mathcal{L}(\hat{y}, y)$ denotes the **loss** on one example, and $J(\boldsymbol{\theta})$ the **objective** to be minimized — the mean loss, possibly augmented by penalty terms.
- $P(\mathbf{x}, y)$ is the unknown **data-generating distribution**; $\mathbb{E}[\cdot]$ denotes expectation and σ^2 a noise variance.
- Later lectures will add η for the learning rate, λ for the regularization strength, and bracketed layer indices $[l]$.
- A hat always means "estimated from data", as in $\hat{y}$, $\hat{f}$, and $\hat{R}$.

Programs written vs. programs learned



Géron (2025), Figs. 1-1, 1-2: a spam filter, twice.

- In the traditional approach (top), humans study the problem and write the classification rules themselves; the program merely applies them.
- In the machine-learning approach (bottom), humans specify a model family and an evaluation procedure, and the rules themselves are extracted from data by training.

A definition of learning

Definition (Mitchell, 1997)

A program **learns** from experience E with respect to task T and performance measure P if its performance at T, measured by P, improves with E.

The definition names the three ingredients that every machine-learning problem must specify:

- the **task** specifies what the program must output — a class label, a real number, a sentence;
- the **experience** is the data available to the learner, drawn from some unknown distribution $P(\mathbf{x}, y)$;
- the **measure** is a number quantifying how well the program performs, fixed *before* training. A whole act of this lecture is devoted to choosing it well.

Everything else in the field — models, losses, optimizers — is machinery for turning experience into performance.

The supervised learning problem, stated

We are given examples $(\mathbf{x}^{(i)}, y^{(i)})$, $i = 1, \ldots, m$, drawn independently from an unknown distribution $P(\mathbf{x}, y)$, together with a model family $\{f_{\boldsymbol{\theta}} : \boldsymbol{\theta} \in \mathbb{R}^p\}$ and a loss $\mathcal{L}(\hat{y}, y)$.

- The quantity we *wish* to minimize is the **risk**, the expected loss on new data:

$$R(\boldsymbol{\theta}) = \mathbb{E}_{(\mathbf{x},y)\sim P}\left[\mathcal{L}(f_{\boldsymbol{\theta}}(\mathbf{x}), y)\right]$$

- The quantity we *can* compute is the **empirical risk**, the average loss on the training data:

$$\hat{R}(\boldsymbol{\theta}) = \frac{1}{m}\sum_{i=1}^{m} \mathcal{L}\left(f_{\boldsymbol{\theta}}(\mathbf{x}^{(i)}), y^{(i)}\right)$$

- **Learning** consists in choosing $\hat{\boldsymbol{\theta}} = \arg\min_{\boldsymbol{\theta}} \hat{R}(\boldsymbol{\theta})$, a strategy called empirical risk minimization. The subject of this lecture is the discipline that makes a low $\hat{R}$ imply a low R.

The difference $R(\hat{\boldsymbol{\theta}}) - \hat{R}(\hat{\boldsymbol{\theta}})$ is called the **generalization gap**.

The task taxonomy I: supervised tasks

A task is **supervised** when the data contains the answers: the labels $y^{(i)}$ are given.

	classification	**regression**
output $\hat{y}$	one of K classes	a real number
example	spam / not spam	house price
typical measure	accuracy, F_1, AUC	RMSE, MAE
this lecture	breast-cancer screening	California housing

AUC = area under the curve; RMSE / MAE = root-mean-squared / mean absolute error — all defined in this lecture's metrics act.

- The boundary between the two settings is softer than it appears: many classifiers output a *score* or a probability per class, which is then thresholded. This observation becomes central when we discuss metrics.
- Both of today's datasets return throughout the year as methodological reference points.

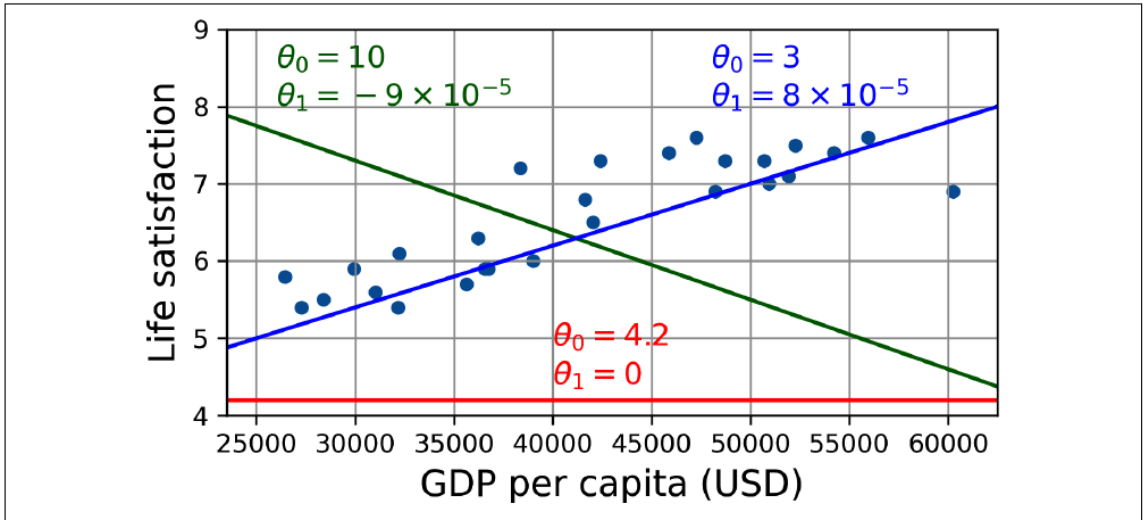
The task taxonomy II: learning without given answers

- In **unsupervised** learning only the inputs $\mathbf{x}^{(i)}$ are given, with no labels at all. The task is to find structure in the data itself: clusters, density estimates, or lower-dimensional representations. The principal component analysis at the end of this lecture is of this kind.
- In **self-supervised** learning the labels are manufactured from the input itself: part of $\mathbf{x}$ is hidden, and the model is trained to predict it from the remainder. No human labeling is required, yet the full supervised machinery applies unchanged. This is how large language models are pretrained, as Lecture 10 will show.

Two neighboring settings, for orientation

- In **reinforcement learning** an agent acts and an environment returns rewards; the data the learner observes depends on its own behavior. This is a separate learning problem with its own theory, covered by the instructor's separate thirteen-lecture RL course; we will only cross-reference it, in Lectures 10 and 13.
- Orthogonal to all of the above is the distinction between **batch** learning, which trains once on a fixed dataset, and **online** learning, which updates the model continuously as data arrives in production. Everything in this course is batch learning; the online setting is an engineering concern that we note and set aside.

Instance-based vs. model-based learning



Géron (2025), Fig. 1-18: three candidate linear models for life satisfaction vs. GDP.

- An **instance-based** method memorizes the examples and predicts by similarity to them; k-nearest neighbors is the classic case. There are no parameters to fit.
- A **model-based** method specifies a parameterized family $f_{\boldsymbol{\theta}}$ and fits $\boldsymbol{\theta}$ by minimizing a loss. In the figure the family is the set of all lines, and training selects one of them.
- This course is model-based from beginning to end. A deep network realizes the same picture with a vastly richer family.

The optimism of the training error

For any *fixed* $\boldsymbol{\theta}_0$, the training error is an unbiased estimate of the risk: $\mathbb{E}\big[\hat{R}(\boldsymbol{\theta}_0)\big] = R(\boldsymbol{\theta}_0)$, because it is the mean of independent, identically distributed — **i.i.d.** — losses. What is reported, however, is the error of the *minimizer* $\hat{\boldsymbol{\theta}}$, and minimization breaks unbiasedness:

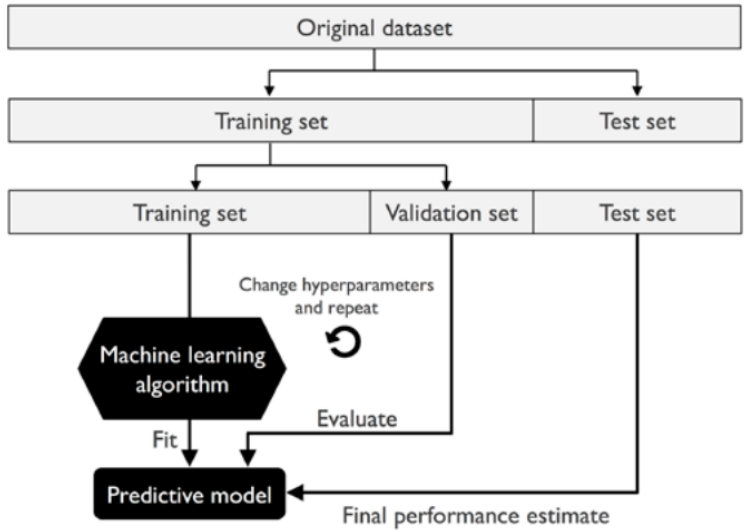
$$\mathbb{E}\Big[\min_{\boldsymbol{\theta}} \hat{R}(\boldsymbol{\theta})\Big] \;\leq\; \min_{\boldsymbol{\theta}} \mathbb{E}\big[\hat{R}(\boldsymbol{\theta})\big] \;=\; \min_{\boldsymbol{\theta}} R(\boldsymbol{\theta}) \;\leq\; R(\hat{\boldsymbol{\theta}}).$$

The first inequality: for every fixed $\boldsymbol{\theta}_0$ we have $\min_{\boldsymbol{\theta}} \hat{R}(\boldsymbol{\theta}) \leq \hat{R}(\boldsymbol{\theta}_0)$; taking expectations gives $\mathbb{E}[\min_{\boldsymbol{\theta}} \hat{R}] \leq R(\boldsymbol{\theta}_0)$, and since this holds for every $\boldsymbol{\theta}_0$, it holds for the best one.

Consequence

The minimized training error is a **downward-biased** estimate of the risk of the selected model. A valid evaluation requires data that the minimization never touched.

The dataset contract: train / validation / test



Raschka et al. (2022), Fig. 6.2.

- The **training set** fits the parameters $\boldsymbol{\theta}$.
- The **validation set** compares models and hyperparameters. Every comparison consumes some of its evidential value, because the chosen model becomes adapted to its particular sample.
- The **test set** is touched **once**, at the very end, and yields the final unbiased estimate. A test set consulted twice has silently become a validation set.

Data leakage I: the preprocessing leak

Leakage denotes any path by which information about the held-out data — or about the future — enters training. It is the most common way in which apparently sound evaluations fail, and it is therefore treated at length here and demonstrated numerically later in the lecture.

- Suppose the dataset is standardized using the mean and variance of *all* rows, and split into train and test afterwards. The training procedure then possesses the test set's statistics, and the test score is no longer an unbiased measurement of performance on unseen data.
- The same error applies to feature selection, where its effect is severe: near the end of this lecture this exact mistake is run on pure noise and observed to produce an apparent accuracy of 88%.

Data leakage II: the identity leak and the time leak

- When several examples share an identity — for instance, several medical images of the same patient split across train and test — a model can score well by recognizing the *patient* rather than the disease, and the test score measures the wrong quantity. The split must keep such groups together on one side of the boundary.
- When the data is ordered in time, a random split allows training on Wednesday's observations to predict Tuesday's. Splits must respect time; the scheme appropriate to temporal data, forward chaining, is presented with the i.i.d. assumption ahead.

Rule

Everything learned from data — scalers, encoders, selected features, and the model itself — is learned from the **training portion only**.

The i.i.d. assumption, and when it breaks

Everything so far has assumed that the examples are i.i.d.: independent draws from one fixed distribution $P(\mathbf{x}, y)$. Both halves of this assumption fail in practice, in different ways.

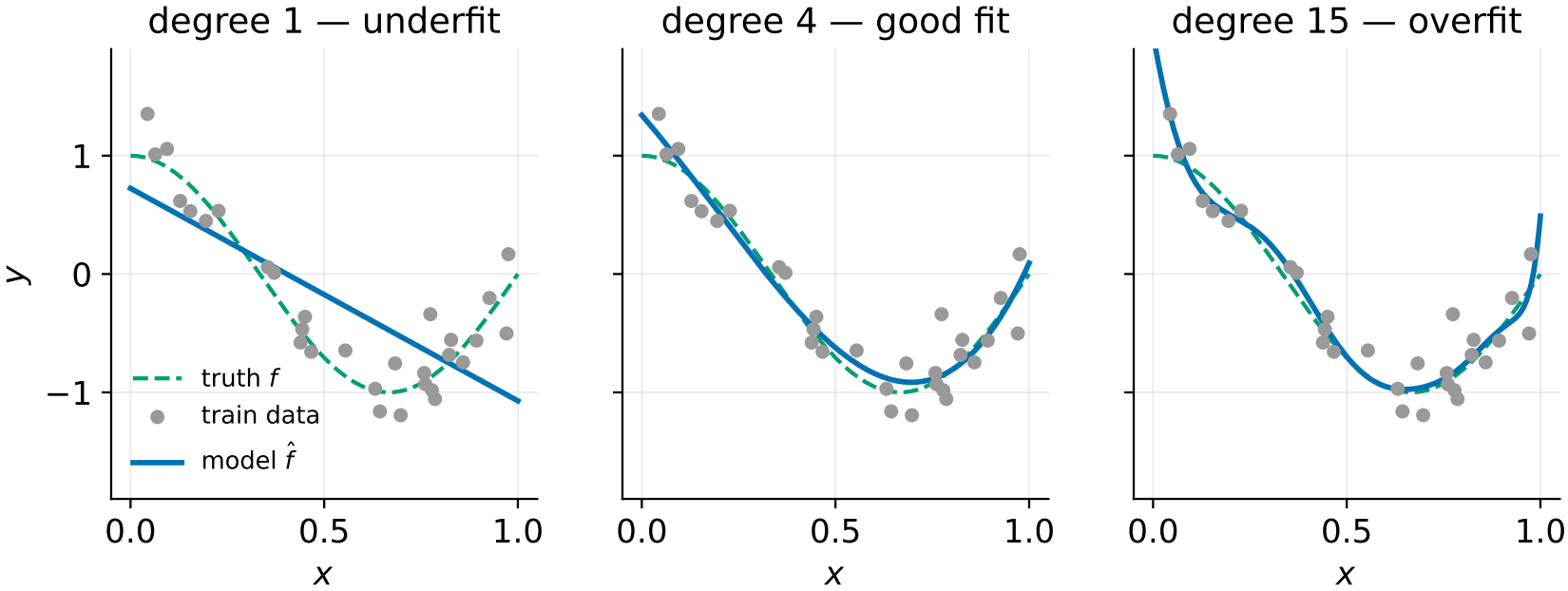
- *Independence* fails for grouped data (the same patient, the same sensor) and for sequential data. The remedy lies in the *split*: group-aware splitting, and for time series **forward chaining**, which always trains on the past and validates on the future:

  `[train 1:100]` $\rightarrow$ `val 101:120;` `[train 1:120]` $\rightarrow$ `val 121:140;` `...`

- *Identical distribution* fails when the world changes between training and deployment — a sensor is recalibrated, user behavior shifts. No split protects against this; it is detected by *monitoring* after deployment, the subject that closes this lecture.

A random shuffle-split of temporal data is precisely the time leak described earlier, in disguise.

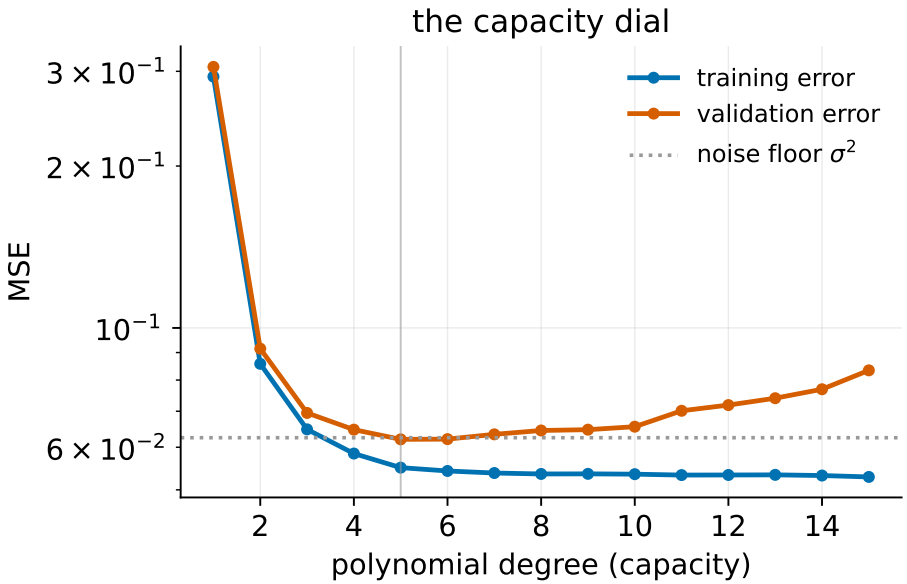
Underfitting and overfitting, visualized



Truth $f(x) = \cos(1.5\pi x)$, noise $\sigma = 0.25$, $m = 30$ points; polynomial fits. Generated by `code/01_generalization.py`.

- The degree-1 family cannot represent f at all. This is **underfitting**: the error originates in a family that is too small, and no amount of additional data will repair it.
- The degree-15 polynomial passes through every noisy point. This is **overfitting**: the model has fitted the *noise*, and the noise will not repeat on new data.

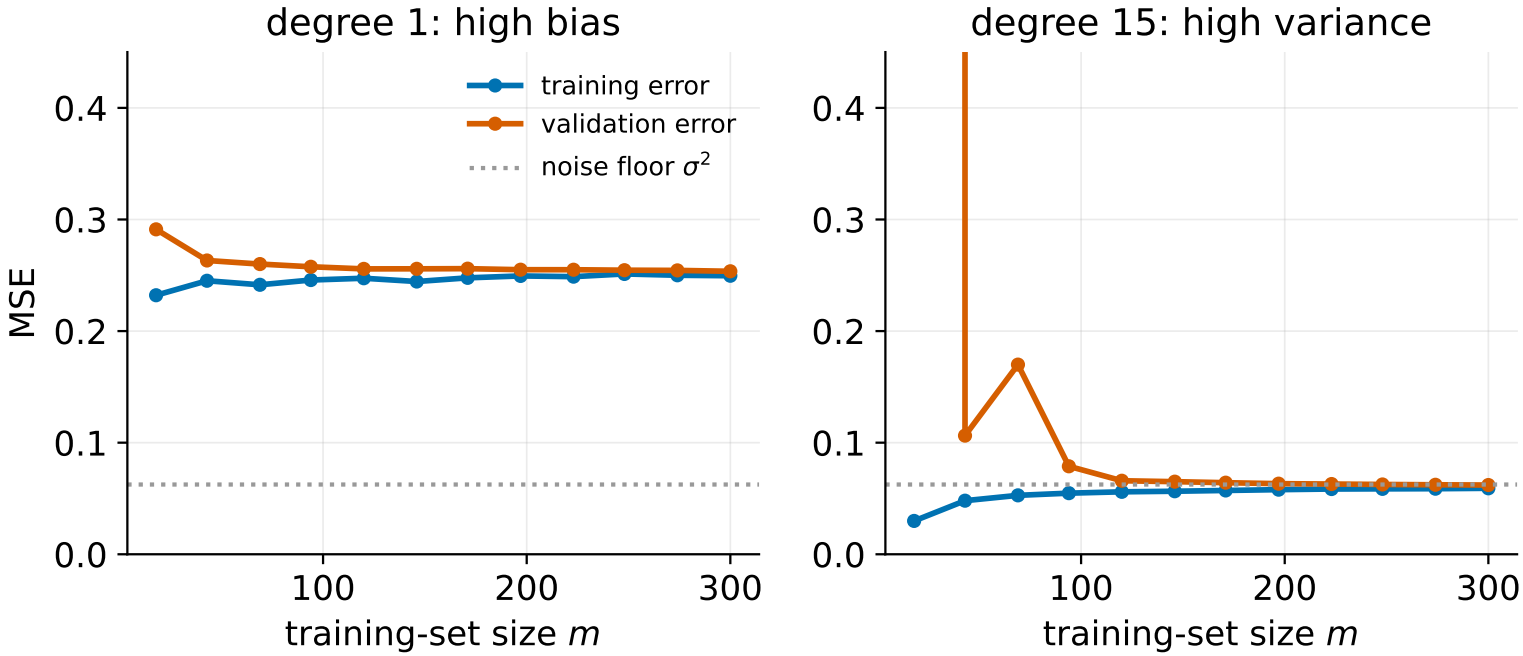
The capacity dial: the validation curve



- The training mean-squared error (MSE) decreases monotonically with capacity: polynomials are nested, so a larger family fits the sample at least as well.
- The validation error is U-shaped: it decreases while added capacity removes bias, and increases once the model begins to fit noise.
- The frozen output of the run reads `best degree = 5, val MSE = 0.0621`, against the noise floor $\sigma^2 = 0.0625$: the optimum attains the best error any predictor can achieve.

No predictor improves on the noise floor — even the true f attains σ^2 on noisy targets. This is proved in the bias–variance derivation ahead.

Learning curves: error as a function of data

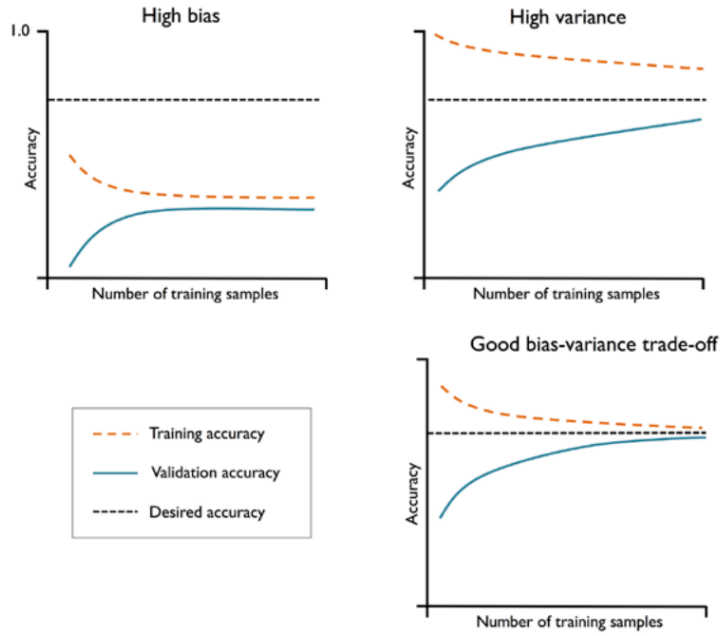


The same truth as before; error vs. training-set size m. Generated by `code/01_generalization.py`.

- For the too-small family (left), both curves converge quickly to a *high* plateau. Additional data will **not** help this model; its error is bias.
- For the too-large family (right), a large gap opens between training and validation error and closes only slowly. Additional data **does** help here — data is the remedy for variance.

Reading learning curves: the diagnosis chart

The schematic form of the two regimes, and the interventions each one calls for:



Raschka et al. (2022), Fig. 6.4.

symptom	diagnosis	interventions
both errors high, small gap	high bias	larger family, better features
train $\ll$ validation	high variance	more data, regularization
both low, small gap	done	stop tuning

The bias–variance decomposition: setup

This is the lecture's central derivation. We fix a test input $\mathbf{x}$ and drop it from the notation, so that f stands for $f(\mathbf{x})$, and so on.

- The data-generating model is $y = f + \varepsilon$ with $\mathbb{E}[\varepsilon] = 0$ and $\mathrm{Var}(\varepsilon) = \sigma^2$. Here f is the true regression function and ε is measurement noise, independent of everything else.
- The training set D — its m points *and their noise* — is itself random, and training on D produces the estimator $\hat{f}_D$.
- The quantity of interest is the **expected squared error on a new observation**,

$$\mathbb{E}_{D,\varepsilon}\left[(y - \hat{f}_D)^2\right],$$

  averaged over both sources of randomness: the realized training set, and the noise on the new point.
- For the average trained model we write $\bar{f} = \mathbb{E}_D[\hat{f}_D]$.

The derivation, step 1: average over the new point's noise

We hold the training set D fixed, so that $\hat{f}_D$ is a constant, and expand $y = f + \varepsilon$:

$$\begin{aligned}
\mathbb{E}_\varepsilon\left[(y - \hat{f}_D)^2\right] &= \mathbb{E}_\varepsilon\left[(f + \varepsilon - \hat{f}_D)^2\right] \\
&= \mathbb{E}_\varepsilon\left[(f - \hat{f}_D)^2 + 2(f - \hat{f}_D)\,\varepsilon + \varepsilon^2\right] \\
&= (f - \hat{f}_D)^2 + 2(f - \hat{f}_D)\,\underbrace{\mathbb{E}[\varepsilon]}_{=0} + \underbrace{\mathbb{E}[\varepsilon^2]}_{=\sigma^2} \\
&= (f - \hat{f}_D)^2 + \sigma^2.
\end{aligned}$$

- The cross term vanishes because the new point's noise has mean zero and is independent of D.
- The term σ^2 remains unchanged. This is the **irreducible noise**: no model, however perfect, removes it, and it is precisely the noise floor observed on the validation curve.

The derivation, step 2: average over training sets

Averaging the first term over D, inserting $\bar{f} = \mathbb{E}_D[\hat{f}_D]$ by adding and subtracting it:

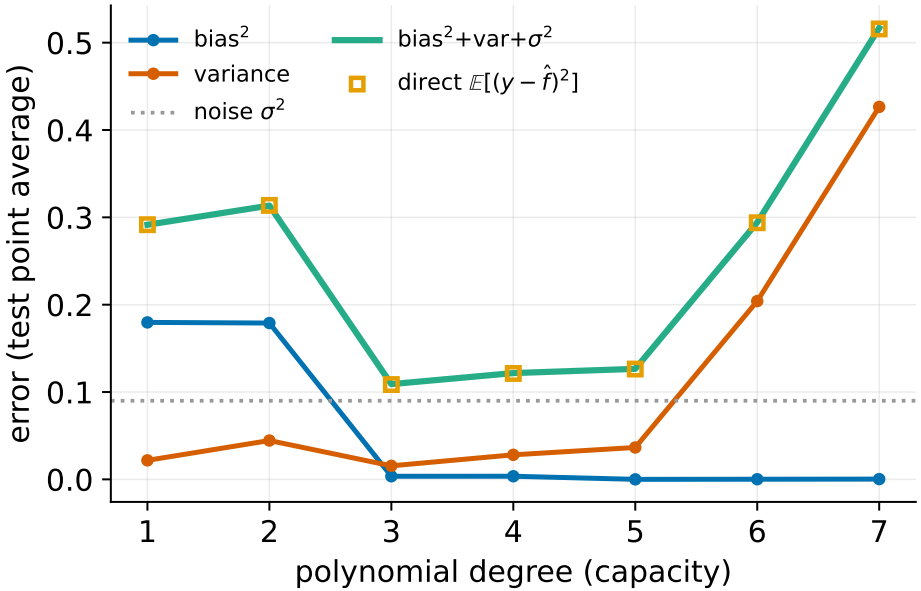
$$\begin{aligned}\mathbb{E}_D\left[(f - \hat{f}_D)^2\right] &= \mathbb{E}_D\left[\left((f - \bar{f}) + (\bar{f} - \hat{f}_D)\right)^2\right] \\ &= (f - \bar{f})^2 + 2(f - \bar{f}) \underbrace{\mathbb{E}_D[\bar{f} - \hat{f}_D]}_{=0} + \mathbb{E}_D\left[(\bar{f} - \hat{f}_D)^2\right] \\ &= (f - \bar{f})^2 + \mathrm{Var}_D\left(\hat{f}_D\right).\end{aligned}$$

The cross term vanishes because $\mathbb{E}_D[\hat{f}_D] = \bar{f}$ by definition. Combining both steps:

Bias–variance decomposition (squared loss)

$$\mathbb{E}_{D,\varepsilon}\left[(y - \hat{f}_D)^2\right] = \underbrace{(f - \bar{f})^2}_{\text{bias}^2} + \underbrace{\mathbb{E}_D\left[(\hat{f}_D - \bar{f})^2\right]}_{\text{variance}} + \underbrace{\sigma^2}_{\text{noise}}$$

The decomposition, verified by simulation



Frozen output (`code/02_bias_variance.py`; 2000 training sets per degree, $m = 30$, $\sigma = 0.3$):

d	bias^2	var	sum	direct
1	.180	.022	.291	.292
3	.004	.015	.109	.109
5	.000	.037	.126	.126
7	.000	.427	.517	.516

(sum $=$ bias^2+var+σ^2, $\sigma^2 = 0.09$)

```
max relative |sum - direct| =
0.0046
PASS
```

The three derived terms, estimated independently, sum to the directly measured expected error: the derivation is confirmed by the simulation.

Reading the three terms

- The **bias** term $(f - \bar{f})^2$ measures how far the *average* trained model lies from the truth. It is the error the family commits even under unlimited retraining — the signature of a family that is too small, or systematically drawn away from f.
- The **variance** term measures how strongly the fit varies from one training set to the next. A high-capacity model follows its particular sample closely, so different samples produce visibly different fits.
- The **noise** term σ^2 is attributable to no model. It bounds every predictor from below, which is why the validation curve plateaued at σ^2 rather than at zero.
- A small family is dominated by bias, a large family by variance. The U-shaped validation curve is precisely the sum of these two contributions above the constant σ^2.

The regularization ledger, and a caveat

- Every anti-overfitting tool in this course is a **variance-reduction** device that incurs a — preferably small — bias cost: the regularization penalties of Lecture 2 (ridge and lasso), the weight decay, dropout, and early stopping of Lecture 6, the data augmentation of Lecture 7, and additional data itself.
- We will call this trade-off *the ledger*: what a method costs in bias against what it saves in variance. The ledger is invoked throughout the year, whenever a new regularizer appears.
- One caveat is required. For heavily over-parameterized deep networks the classical U-shape is not the complete picture — the phenomenon known as "double descent", which we name but do not treat. The decomposition itself remains exactly true; it is the *capacity* axis that becomes subtle. We return to this in Lectures 5 and 6.

Accuracy, and its failure under imbalance

Accuracy is the fraction of correct predictions. As a worked numeric (verified in `code/verify_numerics.py`), consider screening data with 1000 patients, of whom 10 are ill — a prevalence of 1%.

- The classifier that declares *every* patient healthy attains

$$\text{accuracy} = \frac{990}{1000} = 0.99, \qquad \text{while identifying } \textbf{none} \text{ of the ill.}$$

- A model can therefore be 99% accurate and clinically useless: on imbalanced data, accuracy largely measures the imbalance itself.

Baselines first

Before any score is credited, the trivial baseline is computed — `DummyClassifier` predicts the majority class, `DummyRegressor` the mean. A model that does not surpass it has learned nothing; a dummy baseline reappears in the housing project.

The confusion matrix

Our binary convention: **positive denotes the class of primary interest** — in a screening task, the malignant case. The confusion matrix tabulates predictions against reality:

	predicted $-$	predicted $+$
actually $-$	TN true neg.	FP false pos.
actually $+$	FN false neg.	TP true pos.

- The two error types are *different mistakes* with different costs: a false positive is an unnecessary biopsy, a false negative is a missed cancer. Every metric of this act is a different way of weighing these four cells.
- For K classes the same table becomes $K \times K$, with per-class metrics averaged — the "macro" and "micro" averages used from Lecture 3 onward.

The screening example, evaluated

The act's running example is breast-cancer screening: 569 tumors, 30 features, and a scaled logistic regression — its internals are treated in Lecture 2; here it is treated as a score-producing classifier — evaluated on 143 held-out cases. The frozen output of `code/03_metrics.py` reads:

```
confusion matrix [ [TN FP] [FN TP] ] = [[89 1] [4 49]]
accuracy = 0.9650
```

- Reading the four cells: 89 benign tumors were correctly cleared, 1 benign tumor was flagged (an unnecessary biopsy), 4 malignant tumors were missed, and 49 were detected.
- The accuracy of 0.9650 compresses all four numbers into one — and hides the distinction between the 1 unnecessary biopsy and the 4 missed cancers. The next metrics recover it.

Precision, recall, F_1

$$\text{precision} = \frac{\text{TP}}{\text{TP} + \text{FP}} \qquad \text{recall} = \frac{\text{TP}}{\text{TP} + \text{FN}}$$

Precision is the fraction of *flagged* cases that are truly positive; recall is the fraction of *truly positive* cases that are flagged.

As a worked numeric we take the frozen test matrix, with $\text{TN} = 89$, $\text{FP} = 1$, $\text{FN} = 4$, $\text{TP} = 49$:

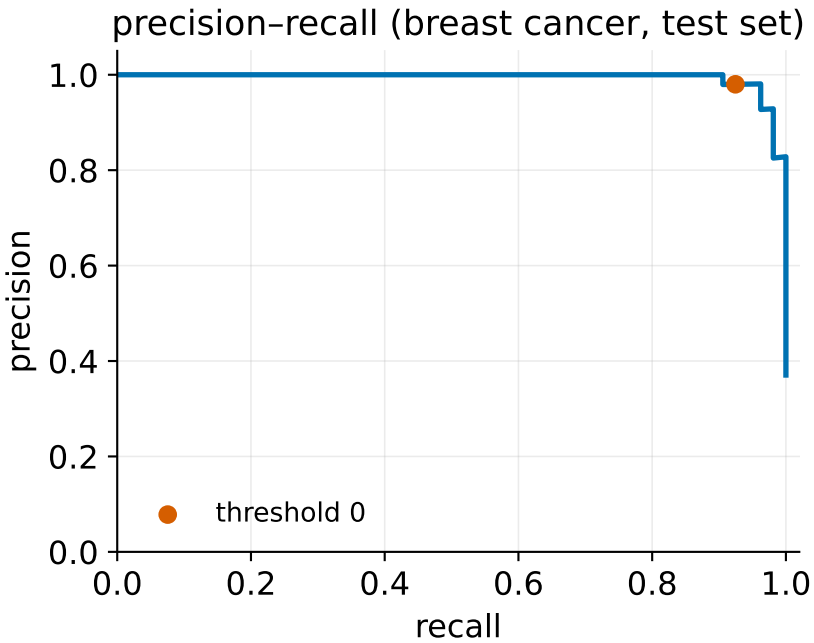
$$\text{precision} = \frac{49}{49 + 1} = 0.980, \qquad \text{recall} = \frac{49}{49 + 4} = \frac{49}{53} = 0.925.$$

The **$\mathbf{F_1}$ score** is their harmonic mean, which is high only when *both* are high:

$$F_1 = \frac{2\,\text{precision} \cdot \text{recall}}{\text{precision} + \text{recall}} = \frac{2 \cdot 0.980 \cdot 0.925}{0.980 + 0.925} = 0.951.$$

All three values are verified against scikit-learn in `code/verify_numerics.py`.

Precision vs. recall: the threshold trade-off



- The classifier outputs a score, and the hard label is obtained by thresholding it: $\hat{y} = \mathbf{1}[\text{score} > t]$, where $\mathbf{1}[\cdot]$ equals 1 if the condition holds and 0 otherwise.
- Raising the threshold flags fewer cases, so precision rises while recall falls; lowering it does the reverse. One trained model therefore yields an entire curve of operating points — the **precision–recall (PR) curve**.
- The operating point is chosen from the *costs* of the two error types — a missed cancer against an unnecessary biopsy — and not from the default threshold.
- The dot marks our test-set model at the default threshold.

The ROC curve, computed by hand

The **ROC** curve (receiver operating characteristic) sweeps the threshold t and plots

$$\mathrm{TPR} = \frac{\mathrm{TP}}{\mathrm{TP} + \mathrm{FN}} \ (= \text{recall}) \qquad \text{against} \qquad \mathrm{FPR} = \frac{\mathrm{FP}}{\mathrm{FP} + \mathrm{TN}}.$$

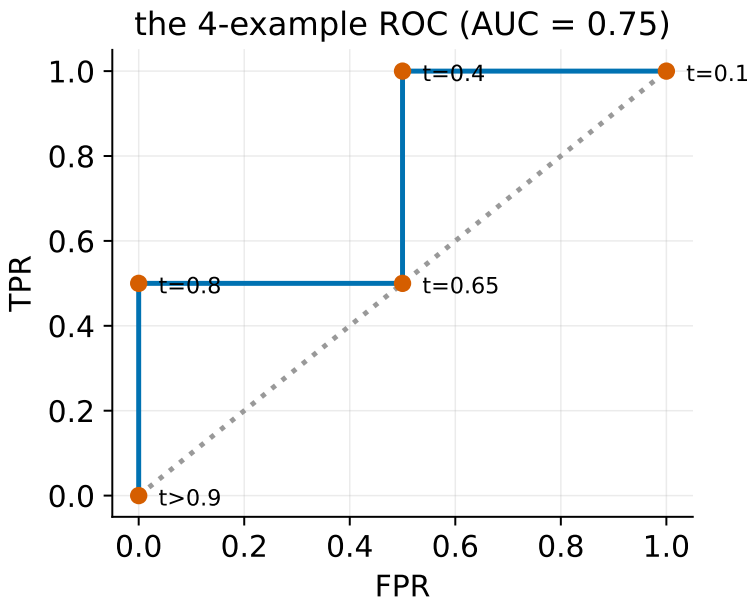
As a worked numeric we take four examples with the following scores and true labels:

example	A	B	C	D
score	0.9	0.7	0.6	0.2
label	$+$	$-$	$+$	$-$

threshold	FPR	TPR
$t > 0.9$: flag nobody	0	0
$t = 0.8$: flag A	0	$1/2$
$t = 0.65$: flag A, B	$1/2$	$1/2$
$t = 0.4$: flag A, B, C	$1/2$	1
$t = 0.1$: flag all	1	1

Each row counts the true positives among the flagged cases (A and C are the two positives) and the false positives among them (B and D are the two negatives). The curve steps through the examples one at a time: each $+$ moves it upward, each $-$ moves it to the right.

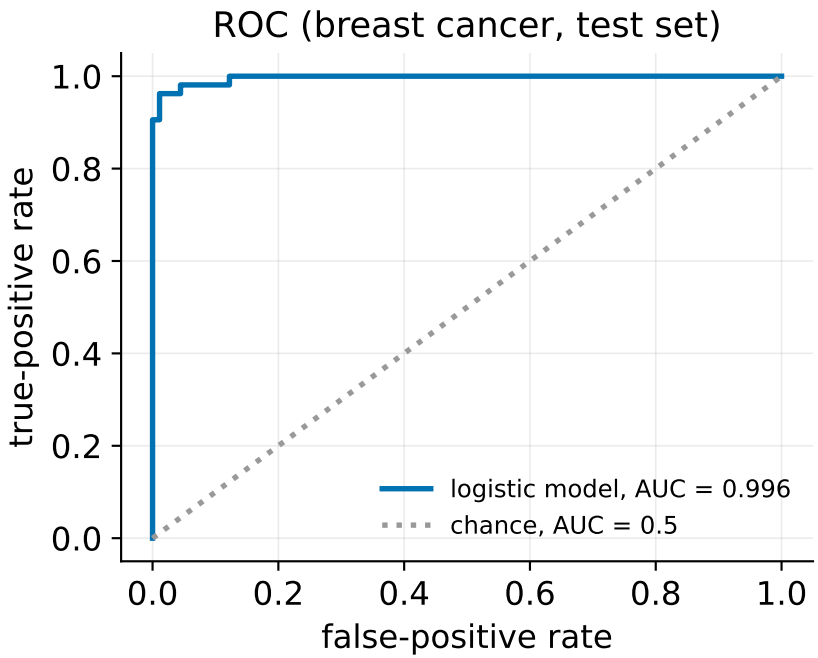
AUC: one number, two readings



AUC is the area under the ROC curve, and it admits two equivalent readings.

- By trapezoids on our staircase, the area consists of two rectangles: $\frac{1}{2}\cdot\frac{1}{2} + \frac{1}{2}\cdot 1 = \mathbf{0.75}$.
- Equivalently, AUC is the probability that a random positive scores higher than a random negative: of our pairs, (A, B), (A, D) and (C, D) are ranked correctly and (C, B) is not, giving $3/4 = \mathbf{0.75}$.
- Both readings are verified against scikit-learn: `AUC = 0.7500 PASS`.
- An AUC of 0.5 corresponds to random ranking, 1.0 to a score under which every positive outranks every negative. AUC is threshold-free: it evaluates the *score*, not one operating point.

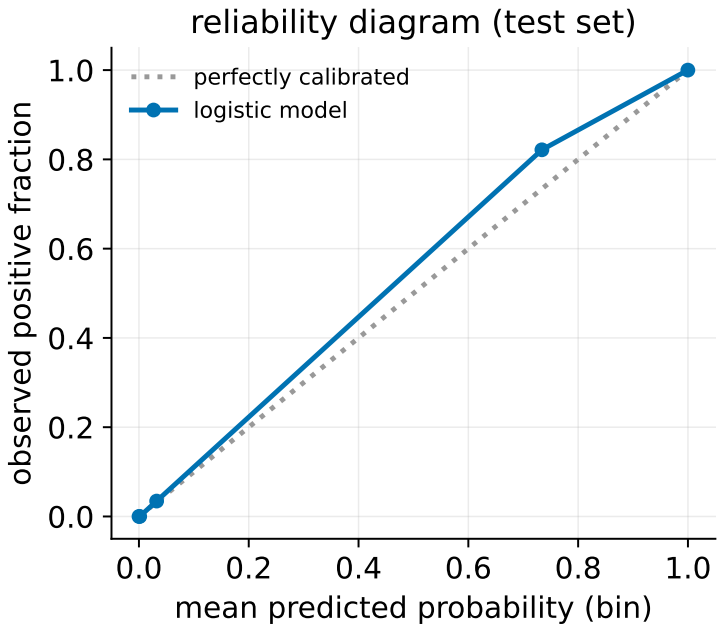
ROC in practice, and when the PR curve is preferable



- Our screening model attains `test-set ROC AUC = 0.9962`: the score ranks nearly every malignant case above every benign one.
- A caution applies under heavy class imbalance. The FPR divides by the large negative count, so an operationally serious number of false positives can still register as a small FPR. Precision divides by the number of flagged cases, and therefore registers the excess immediately.
- In practice the PR curve and F_1 are reported when the positive class is rare and the costs concentrate there; for balanced classes and ranking quality, ROC and AUC are the standard.

Calibration of predicted probabilities

Scores are frequently interpreted as probabilities. A model is **calibrated** if, among cases assigned $\hat{p} \approx 0.8$, about 80% are actually positive.



- The **reliability diagram** bins the test predictions and plots the observed positive fraction against the mean predicted probability. A calibrated model traces the diagonal.
- In the frozen run, the low bins lie on the diagonal, while the 0.734 bin observes 0.821 — the model is mildly underconfident there.
- The **Brier score** condenses the picture into one number: $\frac{1}{m}\sum_i (\hat{p}^{(i)} - y^{(i)})^2 = 0.0224$ here, where 0 is perfect.

Calibration is a separate axis of quality

- A model can rank flawlessly — an AUC near 1 — while its probabilities are far off: any monotone distortion of the scores leaves the ranking, and hence AUC, unchanged.
- Calibration matters whenever thresholds encode costs: a decision rule computed from miscalibrated probabilities inherits the miscalibration, and the expected costs it optimizes are not the real ones.
- Two standard remedies exist, which we name but do not derive: **Platt scaling** and **isotonic regression**. Both are themselves fitted on validation data — the dataset contract applies to recalibration as well.

Regression metrics: RMSE and MAE

The **root-mean-squared error** and the **mean absolute error** are defined by

$$\mathrm{RMSE} = \sqrt{\frac{1}{m}\sum_i (\hat{y}^{(i)} - y^{(i)})^2} \qquad \mathrm{MAE} = \frac{1}{m}\sum_i |\hat{y}^{(i)} - y^{(i)}|.$$

As a worked numeric (verified in `code/verify_numerics.py`), take targets $(2, 3, 5, 4, 10)$ and predictions $(2.5, 3, 4, 4, 4)$. The errors are $(0.5, 0, 1, 0, 6)$, so $\sum |e_i| = 7.5$ and $\sum e_i^2 = 37.25$, giving

$$\mathrm{MAE} = \frac{7.5}{5} = 1.5, \qquad \mathrm{RMSE} = \sqrt{\frac{37.25}{5}} = 2.73.$$

The two numbers disagree substantially on the same predictions — the comparison that follows explains why, and what the disagreement means.

Reading RMSE against MAE

- The square makes RMSE *outlier-sensitive*: in the worked numeric, the single error of 6 contributes 97% of the sum under RMSE's square root, while it contributes 80% of MAE's sum.
- Choosing between the two therefore decides which errors are penalized most. RMSE penalizes large errors disproportionately; MAE counts all errors linearly. Neither is "more correct" — they encode different costs.
- A forward reference to Lecture 2: minimizing squared error is maximum likelihood under Gaussian noise, and minimizing absolute error is maximum likelihood under Laplace noise. Error metrics implicitly encode noise models, and both statements will be derived.

The metric is fixed before training

- The metric is the **contract** with whoever uses the system: it encodes the costs — a missed cancer against an extra biopsy — and it must be fixed *before* any results are seen. Selecting the most favorable metric post hoc invalidates the evaluation.
- The metric is not the loss. The loss is the *differentiable surrogate* that the optimizer descends — accuracy has zero gradient almost everywhere, a statement Lecture 2 makes precise — while the metric is what is reported. The two should be aligned in intent; they rarely coincide in form.
- Every experiment in this course states its metric up front. The Fashion-MNIST "accuracy ledger" that begins in Lecture 3 uses one fixed metric across all of its entries precisely so that its numbers remain comparable across the year.

A single split is a noisy measurement

- A single validation split produces *one* number, and that number fluctuates: a different split yields a different value. The smaller the dataset, the larger the fluctuation.
- The effect is easy to measure. In the frozen run of `verify_numerics.py`, a 60-example subsample of the cancer data (malignant fraction 0.45) is split in half 200 times. The malignant fraction of the held-out half ranges over

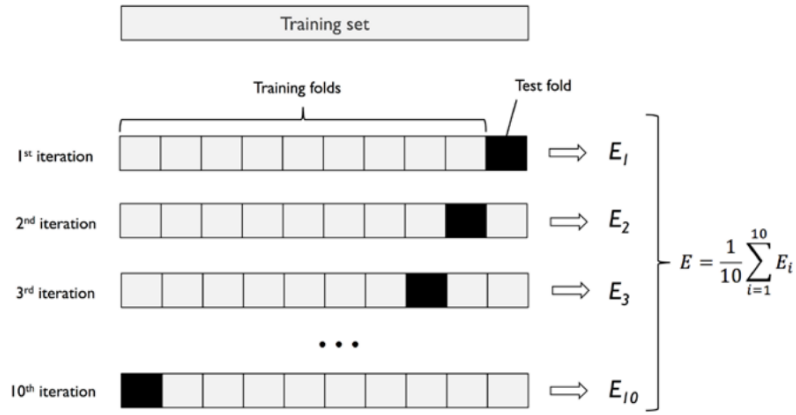
  $0.23 – 0.63$ unstratified vs. $0.43 – 0.47$ stratified.

- An evaluation set can therefore, by pure chance, contain half or double the true prevalence — and scores measured on such sets are not comparable with one another.

Stratification, and the remedy for score noise

- **Stratified splitting** preserves the class proportions in every part of the split. It is the default for classification splits throughout this course, and it is what the 60-example subsample experiment demonstrates.
- Stratification removes the *composition* noise of a split; it does not remove the noise of the *score* measured on it. The two problems have complementary remedies.
- For the score's fluctuation, the remedy is averaging over many splits — **cross-validation (CV)**, which we now develop.

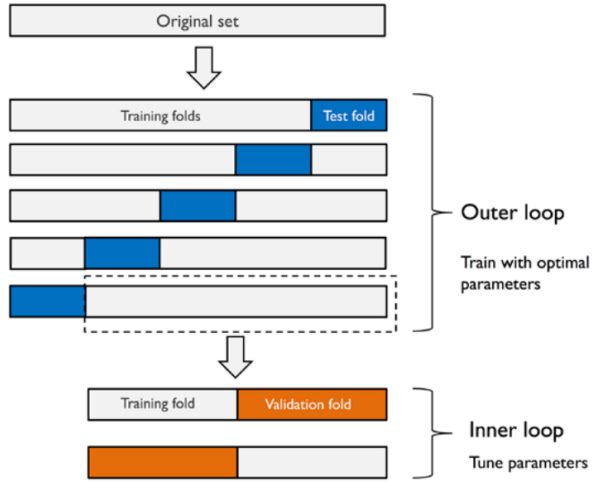
k-fold cross-validation



Raschka et al. (2022), Fig. 6.3.

- The *training* data is split into k folds. The model is trained on $k-1$ of them and scored on the held-out fold; the folds rotate, and the report is the mean $E = \frac{1}{k}\sum_{i=1}^{k} E_i$ together with its spread.
- Every example is scored exactly once, and never by a model that trained on it, so the estimate uses all the data without violating the contract.
- The cost is k trainings; common choices are $k = 5$ or $k = 10$, and the extreme case $k = m$ (leave-one-out) is reserved for very small datasets. For classification, the folds are stratified.

Nested cross-validation: evaluating the tuned procedure



Raschka et al. (2022), Fig. 6.8.

- Tuning *selects* the hyperparameter with the best CV score, and selection biases that score downward. This is the optimism argument from the beginning of the lecture, repeated one level up.
- The remedy is an **outer** loop whose test folds the tuning never sees. The inner loop tunes; the outer loop measures the *entire tuned procedure*.
- In practice, with plentiful data, the train/validation/test split plays the same role. Nested CV is the version required when data is scarce.

Preprocessing I: features on a common scale

Features are measured in widely different units — room counts between 1 and 10, incomes between 10^4 and 10^5. Two standard maps bring them onto a common scale, with the *statistics learned on the training data only*:

$$\text{standardization: } x' = \frac{x - \mu_{\text{train}}}{s_{\text{train}}} \qquad \text{min–max: } x' = \frac{x - x_{\min}}{x_{\max} - x_{\min}}$$

- Standardization, which produces mean 0 and variance 1, is this course's default. Min–max scaling compresses the feature into $[0, 1]$ but is sensitive to outliers, since a single extreme value determines the denominator.
- Trees are indifferent to scale, but **no gradient-trained model is**: unscaled features turn the loss surface into a badly conditioned ravine. Lecture 2 makes this precise for gradient descent, and Lecture 5 returns to it for deep networks. Scaling is not optional; it is part of what makes optimization work.

Preprocessing II: encoding categorical features

- Models consume numbers, so a categorical feature such as `ocean_proximity` $\in \{$INLAND, NEAR BAY, $\ldots\}$ must be encoded. The standard encoding is **one-hot**: one indicator column per category, with a single 1 marking the category present.
- The standard error to avoid is integer-coding a *nominal* feature — setting INLAND $= 0$, NEAR BAY $= 1$, and so on. This invents an ordering and a distance that do not exist in the data, and a model will exploit both.
- Integer codes are legitimate only for genuinely *ordinal* scales, where the order is real — clothing sizes S $<$ M $<$ L, for example.

Preprocessing III: missing values, and the estimator API

- For missing values there are three defensible options: drop the affected rows, drop the affected feature, or **impute** — fill the gaps in. The default imputation is the feature's *training-set* median; optionally, an added "was missing" indicator column lets the model use the missingness itself as signal.
- Scaling, encoding, and imputation are all *learned* transformations, and scikit-learn's estimator API makes this explicit: `fit` learns the statistics — medians, category lists, μ and s — *on the training data*, and `transform` applies them anywhere.
- This is the dataset contract acting at the preprocessing level: whatever is learned is learned from the training portion only.

The leakage-proof tool: `Pipeline`

A `Pipeline` chains transformations and a final model into *one estimator*, and a `ColumnTransformer` routes columns to the appropriate branch. The following is taken from the housing project (`code/05_end_to_end.py`):

```
num_pipe = make_pipeline(SimpleImputer(strategy="median"),
                         StandardScaler())
prep = ColumnTransformer([
    ("num", num_pipe, numeric_columns),
    ("cat", OneHotEncoder(handle_unknown="ignore"),
            ["ocean_proximity"]),
])
model = make_pipeline(prep, RandomForestRegressor())
model.fit(X_train, y_train)     # ALL fitting happens here
```

- Calling `fit` on the training data fits *everything* on the training data, and inside cross-validation each fold refits the entire chain on its own training part. Leakage becomes structurally impossible rather than a matter of discipline.

Leakage, demonstrated numerically

The setup (`code/04_leakage.py`): $m = 100$ examples, 5000 *pure-noise* features, and labels drawn by fair coin flips, independent of the features — so the true attainable accuracy is 0.5. The procedure selects the 20 features most correlated with the labels and then 5-fold cross-validates a linear classifier.

```
WRONG  (select on all data, then CV):  accuracy = 0.880
RIGHT  (selection inside the CV loop): accuracy = 0.530
```

- Among 5000 random features, some correlate with any labels *by chance*. Selecting them while observing all rows lets every CV test fold influence which features are kept, so the test folds have leaked into training.
- The result is an apparent 88% accuracy on coin flips. This exact mistake has produced published results — Ambroise & McLachlan (2002) documented it across gene-expression studies. The pipeline version is sound because the selection happens inside each fold.

Hyperparameter search: grid and random

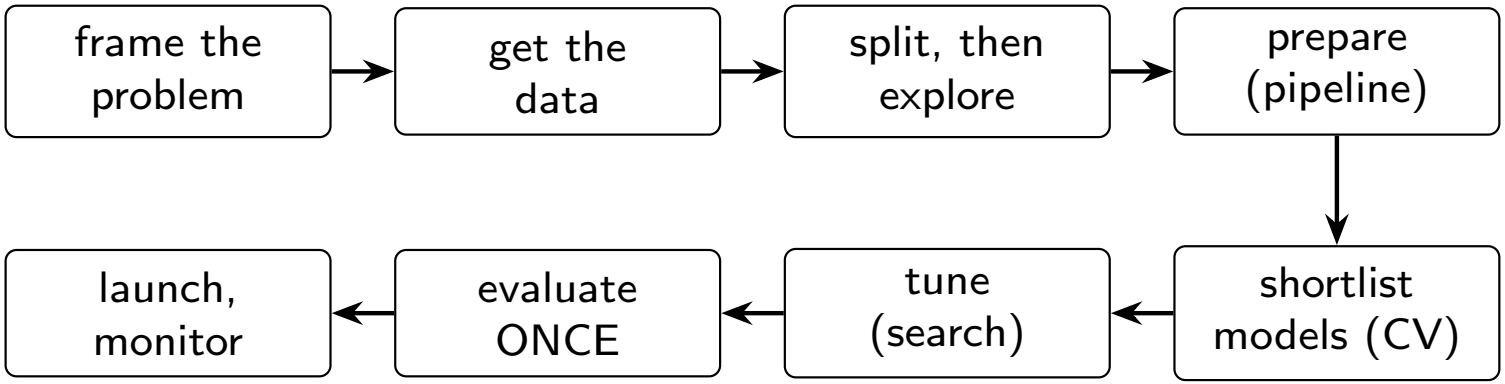
Hyperparameters are the choices not fitted by training itself: tree depth, regularization strength λ, learning rate η. They are searched on *validation* data, and never on the test set.

- **Grid search** tries every combination of the listed values. It is exhaustive, but its cost grows exponentially with the number of hyperparameters.
- **Random search** samples configurations at random. A dated fact: Bergstra and Bengio (2012) showed that it dominates grids when only a few hyperparameters matter, because a grid allocates its trials to re-testing the unimportant axes while random search probes every axis at every draw. Random search is this course's default.

Search practice: scales and budgets

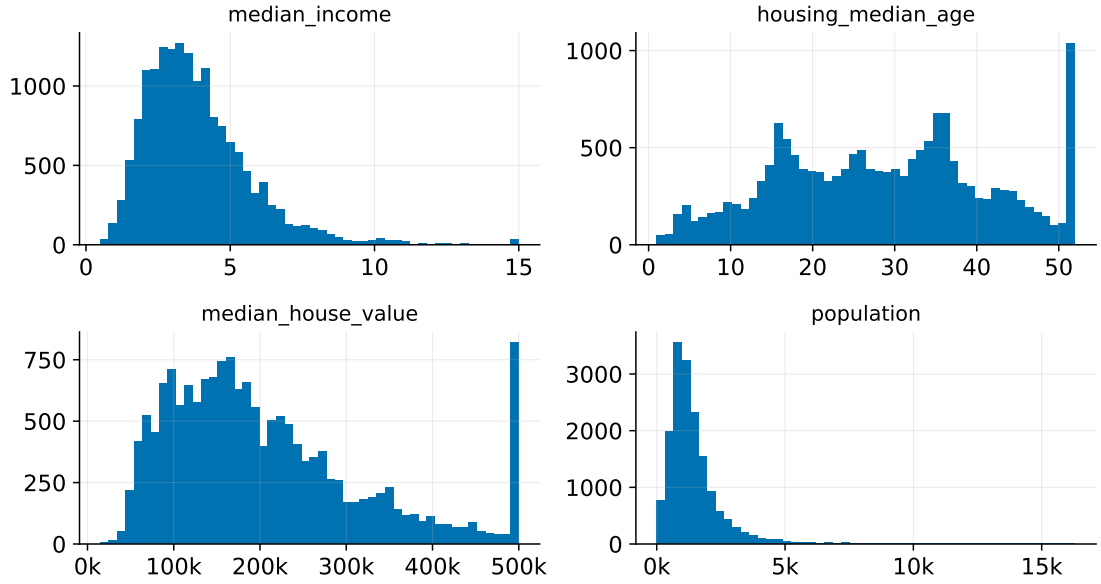
- Scale-type hyperparameters such as η and λ are searched on a logarithmic scale $(10^{-4}, 10^{-3}, \dots)$ — **the log-scale rule** — because the difference between 0.0001 and 0.001 matters as much as the difference between 0.01 and 0.1.
- One resource-efficient variant is **successive halving**: it allocates a small budget to many candidates, discards the weaker half, and doubles the budget for the remaining candidates (`HalvingRandomSearchCV` in scikit-learn).
- Whatever the method, every score consulted during the search is a validation score. The number finally reported comes from the untouched test set — the contract, once more.

The end-to-end workflow



- This is Géron's Chapter-2 project compressed to its essential steps. The workflow is now applied to real data, with every number shown.
- The task is to predict a California district's median house value from census features — a regression problem. Following our own rule, the metric is fixed before any model is trained: **RMSE**.

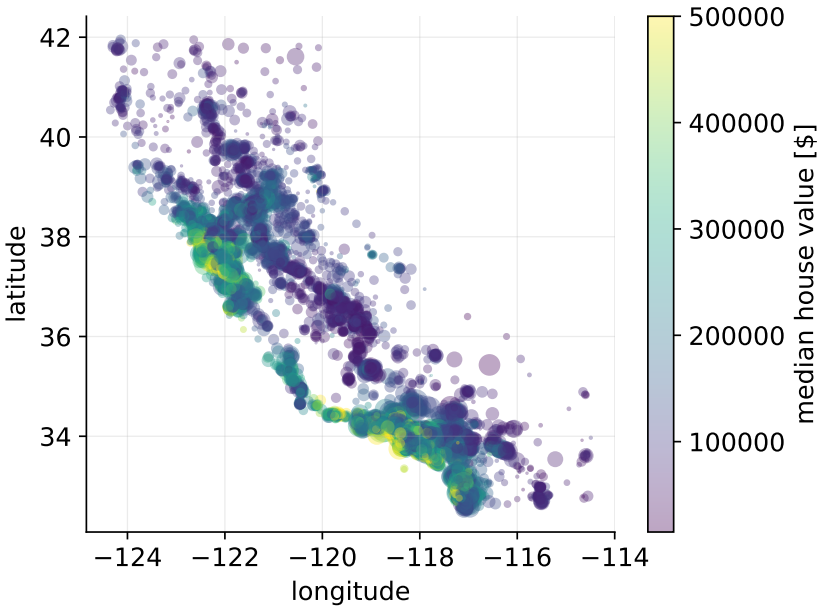
Obtaining and inspecting the data




```
dataset: 20640 districts x 10 columns; missing total_bedrooms: 207
```

- The columns comprise 8 numeric features, 1 categorical feature (`ocean_proximity`), and the target (`median_house_value`); 207 rows have a missing bedroom count to impute.
- The training-part histograms reveal skewed features and capped values — the bar at $500k indicates that the data was preprocessed before it reached us. Inspection precedes modeling.

The split precedes exploration



The split happens *before* exploration, because insights gleaned from test rows would leak into the modeling choices:

```
split: 16512 train / 4128 test
(stratified on income)
```

We stratify on binned income — the feature most correlated with the target — so that both halves receive the same income distribution. This is the stratification argument of the cross-validation act, applied in practice:

```
correlation with target:
median_income      0.69
total_rooms        0.14
housing_median_age 0.10
latitude          -0.14
```

The map shows that geographic position carries structure: coastal prices are higher, a fact no single column carries alone.

Shortlisting models with cross-validation

The same preprocessing pipeline feeds every candidate, so the comparison is fair. The frozen 5-fold CV output of `code/05_end_to_end.py`:

```
dummy (mean)          CV RMSE =  115307 +- 1503   train RMSE =  115311
linear regression     CV RMSE =   68322 +-  354   train RMSE =   68233
decision tree         CV RMSE =   68363 +-  974   train RMSE =       0
random forest         CV RMSE =   49410 +-  289   train RMSE =   18058
```

This table exhibits the lecture's principles in one place; it is read row by row.

Reading the shortlist

- The **dummy** regressor, which always predicts the mean, sets the reference level any model must surpass: an RMSE of about 115k.
- **Linear regression** shows training and CV error nearly equal, so it has no variance problem: it has reached the bias ceiling of the linear family.
- The **decision tree** reports a training RMSE of *zero* against a CV RMSE of 68k: it has memorized the training data perfectly and generalizes no better than the linear model — the fit-panel's degree-15 behavior, on real data.
- The **random forest** averages many decorrelated trees, which reduces the variance — an entry in the ledger. It attains the best CV score; the remaining train–CV gap shows that some variance persists.

Tuning, and the single test evaluation

The forest is tuned by random search — 8 candidates, 3-fold CV — and only then evaluated on the test set:

```
randomized search: best {'max_features': 7, 'n_estimators': 152},
                   CV RMSE = 49274
FINAL test RMSE = 49364   (95% CI [46953, 51662])
```

- The test set is evaluated **once**, after all decisions have been made, and the result is reported with an uncertainty interval — a t-interval on the squared errors — rather than as a bare point estimate.
- The CV estimate of 49.3k predicted the test result of 49.4k. The procedure predicted its own out-of-sample error, and this agreement is precisely the property the contract is designed to deliver.
- That evaluation cannot be repeated. Any further change tuned against this test set would silently turn it into a validation set.

Deployment, monitoring, maintenance

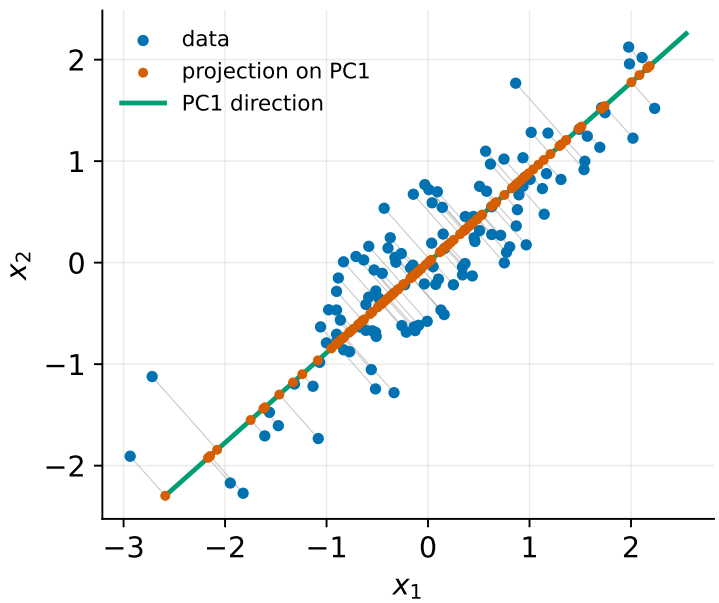
- Deployment is where the i.i.d. assumption ultimately fails: prices drift, demographics shift, sensors change. **Distribution shift** develops over time, and no offline split can rule it out.
- The minimal monitoring obligations in production are to log the inputs and predictions, to track the live metric — or a proxy for it when labels arrive late — and to alert when the input distributions drift away from the training statistics.
- Retraining on fresh data restarts the entire workflow, under the same contract: a new split, the metric unchanged, the test set touched once.
- This course trains models offline throughout, but the contract principle — *the evaluation must predict deployment* — is the reason for today's rules.

A map of the classical model families

family	its one idea	appropriate when
linear models	weighted sum of features	baseline; wide sparse data
k-NN	predict from neighbors	small data, local structure
decision trees	learned if–then splits	interpretability, mixed types
ensembles (forests, boosting)	average / chain many weak models	tabular-data default; strongest classical
support-vector machines (SVMs)	maximum-margin separator	small m, large d

- These families are named so that the surrounding landscape is known; their internals are deliberately not taught here (Géron Ch. 5–8 and Raschka Ch. 3 and 7 treat them thoroughly). On tabular data, tuned ensembles remain the strongest classical baseline — deep learning's principal domain is *perceptual and sequential* data, where the features themselves must be learned.
- Linear models are the exception to the signposting: Lecture 2 builds them completely, because they *are* the neuron.

PCA, I: the projection view



Principal component analysis (PCA) operates on centered data and seeks the unit direction $\mathbf{w}_1$ that captures the most variance,

$$\mathbf{w}_1 = \arg\max_{\|\mathbf{w}\|=1} \frac{1}{m} \sum_{i=1}^{m} \left(\mathbf{w}^\top \mathbf{x}^{(i)}\right)^2,$$

followed by $\mathbf{w}_2 \perp \mathbf{w}_1$, and so on.

- The solution is quoted here rather than derived, since it is standard eigenanalysis: $\mathbf{w}_1, \mathbf{w}_2, \ldots$ are the eigenvectors of the sample covariance matrix, ordered by eigenvalue.
- In the frozen run, the first component explains 93.9% of this cloud's variance, so retaining the single score $\mathbf{w}_1^\top \mathbf{x}$ compresses two numbers into one at a small cost.

PCA, II: the reconstruction view

Projecting onto $\mathbf{w}$ and returning gives the reconstruction $\tilde{\mathbf{x}} = (\mathbf{w}^\top \mathbf{x})\,\mathbf{w}$. Expanding the reconstruction error, using $\|\mathbf{w}\| = 1$:

$$\frac{1}{m}\sum_i \left\|\mathbf{x}^{(i)} - \tilde{\mathbf{x}}^{(i)}\right\|^2 = \frac{1}{m}\sum_i \left(\|\mathbf{x}^{(i)}\|^2 - \left(\mathbf{w}^\top \mathbf{x}^{(i)}\right)^2\right)$$

$$= \text{const} - \underbrace{\frac{1}{m}\sum_i \left(\mathbf{w}^\top \mathbf{x}^{(i)}\right)^2}_{\text{variance along } \mathbf{w}}$$

- The reconstruction error is a constant — the total variance of the data — minus the variance captured along $\mathbf{w}$.
- Minimizing the reconstruction error is therefore equivalent to maximizing the captured variance: the same $\mathbf{w}_1$ emerges from either formulation.

PCA in practice, and a forward reference to Lecture 12

- The frozen run confirms the equivalence numerically: the reconstruction mean-squared error of 0.106 equals the variance that the first component fails to capture.
- Real data often concentrates near a low-dimensional subspace. On the housing data's 8 numeric features, 4 components already carry 96% of the variance (`code/06_pca.py`).

Forward reference to Lecture 12

An encoder–decoder trained on reconstruction error: PCA is the *linear* case, and an **autoencoder** is its nonlinear lift.

Summary — Lecture 1: the contract

- Learning consists in choosing $\hat{\theta}$ by minimizing the empirical risk; the discipline of this lecture lies in making a low $\hat{R}$ imply a low R.
- **The dataset contract** requires that the split come first, that everything be fitted on the training data only, and that the test set be used exactly once. Leakage is the failure mode — we manufactured an apparent 88% accuracy on coin flips to demonstrate it.
- The one derivation to remember is the decomposition

$$\mathbb{E}\big[(y - \hat{f}_D)^2\big] = \text{bias}^2 + \text{variance} + \sigma^2,$$

obtained in two steps and verified by simulation. Every regularizer introduced later is a move on this ledger.

Summary — Lecture 1: the toolbox

- **Metrics** are fixed before training. Accuracy fails under imbalance; precision, recall, F_1, and ROC/AUC were computed by hand; RMSE and MAE encode which errors are penalized; calibration is its own axis of quality.
- The **evaluation tools** are stratified splitting, k-fold cross-validation, and nested cross-validation — the latter evaluating the tuned procedure free of selection bias.
- **The pipeline discipline** demands that scalers, encoders, and imputers be learned on the training data only, and the `Pipeline` makes this structurally impossible to violate.
- **The end-to-end workflow** met its stated criterion: the CV estimate of 49.3k matched the test result of 49.4k — the workflow predicted its own out-of-sample error, which is what the contract is for.

Next lecture: the smallest learning machine

- Today's models were treated as black boxes: we evaluated them without opening them. Lecture 2 opens the simplest one.
- The **perceptron** (1957) is presented first, stated as an algorithm: it is a linear threshold classifier with an error-driven update rule — and it carries a hard built-in limitation that we will prove.
- **Adaline** then makes the decisive change: making the output continuous and the loss differentiable turns classification into *optimization*. Every method in the remainder of the course rests on this change.
- **Gradient descent** is derived in full, producing the update loop

$$\text{forward} \rightarrow \text{loss} \rightarrow \text{backward} \rightarrow \text{step}$$

  on which every model from Lecture 3 to Lecture 13 runs.
- Losses cease to be arbitrary: mean squared error and cross-entropy both follow from maximum likelihood, the principle underlying the diffusion models of Lecture 13.